AR Sudoku: App and Website Design

Muhammad Ali Hassan

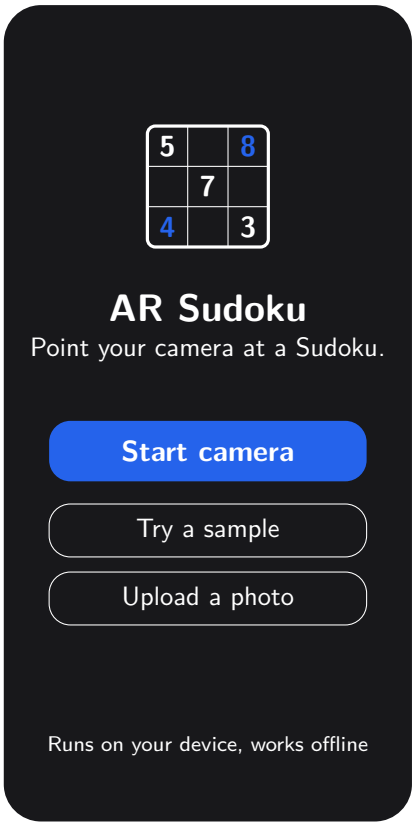
1 Goal

Point any phone or laptop camera at a printed Sudoku. The website finds the puzzle, solves it, and writes the missing digits on the paper. It also works on a puzzle you already started: it checks your handwritten digits and fills in the rest.

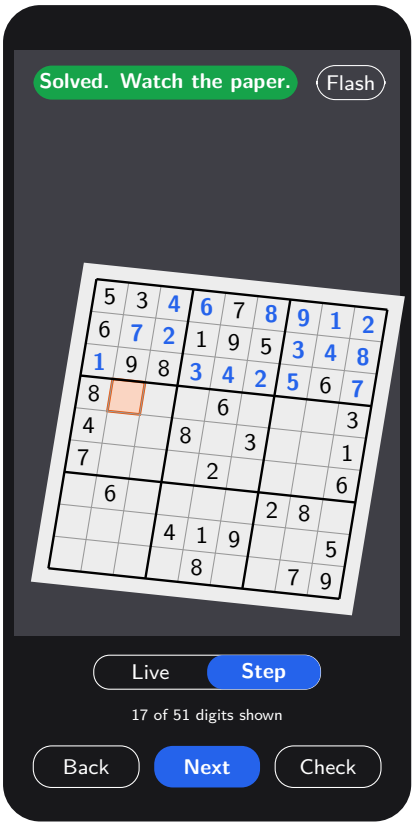
- **Leonardo:** vision and solver (`sudoku_cv`).
- **Me:** the website. Camera, screens, drawing the digits on the paper, free hosting.

Classical computer vision only, no neural networks.

2 Screens



Home screen



Camera screen in Step mode

- **Home:** start the camera, try a sample puzzle, or upload a photo.

- **Camera:** a message on top tells the user what to do. Digits appear on the paper and stay on it when the paper moves. Printed digits are black, the app's digits are blue, the next one is orange.
- **Live or Step:** Live shows all missing digits one after another. Step shows one per tap, so it works as a hint.
- **Check:** if a digit was read wrong, the user freezes the picture and taps to fix it.
- **Continue mode:** the user's handwritten digits get green when right, red when wrong.

What the code reports	Message on screen
no puzzle found	Point the camera at a Sudoku
puzzle found, reading	Hold still…
can't be solved	Some digits look wrong. Tap Check.
solved	Solved. Watch the paper.

3 How it works

1. The camera gives a stream of video frames.
2. Whenever it is free, the site takes the newest frame, shrinks it to 640 px, and gives it to Leonardo's code. His Python runs inside the browser with Pyodide, in a background thread, so the video never freezes.
3. His code returns the puzzle's 4 corners, the digits, the solution and the order to show them.
4. The site writes the digits on a flat square and bends it onto the paper with the homography (a CSS `matrix3d` transform).

There is no server: nothing to pay for, the video never leaves the phone, and the site works offline after the first visit. It is hosted for free on GitHub Pages. My code lives in `app/`.

If this is too slow on phones: use smaller frames (480 px), or track the corners with OpenCV.js, or as a last resort run the Python on a free Hugging Face server.

4 What I need from Leonardo

- A `solve(givens)` function, to solve again after the user fixes a digit.
- For Continue mode: handwritten digits kept apart from printed ones, plus a list of wrong ones. Handwriting can be read classically with HOG features and k-nearest neighbours on MNIST digits.
- A confidence score per cell, so the Check screen can shade doubtful cells.
- Optional: the in-between images (threshold, contour, flat grid) for a "How it works" demo.

5 Steps to finish

	Dates	What I do
1	by Nov 2	Run Pyodide + OpenCV on a laptop, an iPhone and an Android phone and time it. Put a test page on GitHub Pages. Decide if the in-browser plan is fast enough.
2	Nov 3 – 9	Camera on all devices. Send frames to the background thread. Draw the digits from Leonardo's stub onto the sample photos.
3	Nov 10 – 15	All screens: home, messages, Live and Step, Check, Continue colours, photo upload, offline install.
4	Nov 16 – 22	Swap the stub for Leonardo's real code. Test on the sample photos.
5	Nov 23 – 30	Test on the 3 devices, let 5 classmates try it, fix problems. Code freeze Nov 30.
6	Dec 1 – 11	Demo on Dec 1 or 3 with a backup video. Report due Dec 11.

6 Testing

On a laptop, an iPhone and an Android phone:

- **Speed:** digit updates per second (target: 15 on a laptop, 10 on phones).
- **Load time:** under 10 s the first time, under 2 s after.
- **Time to solve:** from "Start camera" to solved, on 10 printed puzzles.
- **Steadiness:** how much the digits shake while the paper is still.
- **Continue mode:** on 10 half-filled puzzles, how many wrong digits are caught.
- **Ease of use:** 5 classmates try it with no help.

7 Risks

- **Too slow on phones:** the fallbacks in section 3.
- **Handwriting read wrong:** the user fixes it on the Check screen.
- **iPhone camera issues:** test on a real iPhone in week 1; photo upload always works.
- **Demo fails in class:** recorded backup video.